openAG — Pipelines (B-ROUND + B1–B7)

Status: Design / seed model. Companion to `2026-10-04-openAG-graph-native-engine-design.md` (the formal core) and `2026-10-04-openAG-entity-model.md` (the gen-0 entity ledger). Date: 2026-10-04

> Revision 2026-10-12 (session ledger fold-in). Four locks propagated into the pipeline view: - B-ROUND reframe — `govern` is now neural (4-step symbolic-legality → symbolic-budget → neural-VoI → symbolic-clamp); `ground` is mixed per-axis (structural→symbolic, semantic→neural) and arbitrates four fact-source oracles (§1). - B0 continuous-ROUND opening — a new §2.0 stakeholder-needs behavior seeds the graph with OPEN promises; no hard B0→B1 boundary (one continuous ROUND, human oracle fades). - B1 = extract-as-fixpoint — Extract is reframed as B-ROUND to promise-fixpoint (top-down intent + bottom-up grounding); `scan` survives only as a code-scan fact source; A1 decomposition is emergent, not a phase (§2.1). - Integration is no longer deferred — it is grounding the dynamic (right-of-Vee) interface promise via the runtime oracle; new `gap:integration-mismatch` (§2.4 note).

---

0. How to read this doc

openAG has one engine — the `B-ROUND` fixpoint — and seven behaviors (`B1–B7`) that are entry points into it. Every behavior either drives rounds (feeds new open promises in: B1/B2/B3/B6), is a round stage run standalone (B7 = gate), or reads the converged graph to emit something (B4/B5).

This doc gives, in order:

1. §1 — B-ROUND, the shared fixpoint, as an ordered pipeline with per-stage purpose + typed I/O, naming the owned I/O entities from entity-model §8.
2. §2 — B1–B7, each as an ordered pipeline (stage · purpose · I/O · realizing component), with the full sub-behavior fan-out that §5 of the entity model summarized as "≈33 behaviors".
3. §3 — the comparison matrix vs. the five existing AMS pipelines (legacy 10-stage, v3 7-stage, walk 6-stage, lifecycle substrate, CAP-DF ceremony), stated per-pipeline as fork / reuse / replace.
4. §4 — the operator crosswalk: openAG `select→propose→ground→admit→reconcile→bind` ↔ walk `select→traverse→classify→rollup→evidence→emit` ↔ the `model.*` L3 functions.

Notation. ⇒ = produces (A5, an owned I/O entity). ← = consumes. Stage purpose is the functional purpose, not an implementation note.

---

1. B-ROUND — the shared fixpoint

`B-ROUND` is a monotone worklist / dataflow framework (design §3.3): expanders are transfer functions, `reconcile` is the join (a closure operator), the budget is the widening operator that guarantees termination. One round is:

```
govern → grow → ground → reconcile → bind → gate
```

Rounds repeat until no open promise remains within budget. Because `reconcile` is a closure operator, the run is confluent modulo budget: scheduling order changes which gaps appear under a budget cut, never the canonical grounded form.

#	stage (BEHAVIOR.*)	purpose	consumes (←)	produces (⇒ owned I/O)	realizing component
1	govern	Neural govern (4 steps): (1) symbolic legality filter — down-only acyclic promise-DAG; (2) symbolic budget read — owns widening + termination; (3) neural prioritization by VoI; (4) symbolic bundle + budget-clamp (free T1 auto-drains emergently; overflow → gap:budget-exhausted). Correctness-neutral: confluence (not govern) is the guarantor; govern is an accelerator.	open-promise worklist, CONparametric budgets, IFLLMProvider (VoI step)	JOB-* (per scheduled discharge)	BudgetGovernor, LLMProvider-Router
2	grow	Select the next open promise; an expander (symbolic or neural) proposes candidate edges/nodes along the promise's axis.	selected Promise, ExpanderRegistry (kind,axis)	candidate set (in-flight, not yet admitted)	ExpansionKernel, ExpanderRegistry, PromiseEngine

#	stage (BEH-ROUND.*)	purpose	consumes (←)	produces (⇒ owned I/O)	realizing component
3	ground	Mixed per-axis check of each candidate against typed evidence — structural axes → symbolic, semantic axes → neural — arbitrating four fact-source oracles (code-scan, human-dialogue, model-self, runtime/composition-execution); reject or downgrade ungrounded candidates to a gap; score confidence.	candidate set, EvidenceRef, A7 signal from all four oracles	Evidence nodes (promoted, deduped)	Grounder
4	reconcile	Closure operator — normalize to canonical form: dedup, identity-bind via θ-crosswalk, roll-up aggregation, canonicalize. The join of the lattice.	grounded candidates, IdentityStore	— (mutates ModelGraph)	Reconciler, IdentityStore
5	bind	Commit stable IDs (sticky across generations) and record the new revision; no silent re-binding.	reconciled graph delta	GEN-* (Generation), DEC-* where a design choice was made	GenerationStore, IdentityStore

#	stage (BEH-ROUND.*)	purpose	consumes (←)	produces (⇒ owned I/O)	realizing component
6	gate	Admit only schema+constraint-valid state; evaluate the 3 verification classes; emit any residual as a typed gap (gap-honesty).	committed graph, CON-existence/prohibited/parametric	JRN-gates + GateEvent; typed gaps/promises	GateEvaluator

> Operator vs round. The design's one-step operator is `select → propose → ground → admit → reconcile → bind` (discharging a single promise). The round `govern → grow → ground → reconcile → bind → gate` is the batched operator: `govern` wraps `select` with a schedule, `grow` = `select`+`propose`, `gate` = batched `admit`. The 1:1 mapping is in §4.

Termination (invariant #2). Every discharge either grounds in finite evidence or emits a gap; budget widening guarantees the worklist empties. Gap-honesty (invariant #5): anything not grounded within budget is emitted as a typed gap, never silently omitted.

> Why neural govern is safe (compatibility principle). `govern` schedules only the down-and-local half of the round — `reconcile` owns all up-flows (roll-up + flow-down verification), which keeps the govern DAG acyclic. Because `reconcile` is a closure operator, scheduling is correctness-neutral: a mis-prioritized VoI step changes which gaps surface under a budget cut, never the canonical grounded form. Govern is an accelerator, not a guarantor. Caveat: the neural VoI step runs at temperature 0 so gap-sets are reproducible. The VoI economic model — `VoI = (value × expected confidence gain) ÷ cost`, termination when `top-VoI < cost-floor` — also yields the new economic gaps `gap:trivial` (neural triviality), `gap:low-confidence` / `gap:confidence-ceiling` (mixed σ+neural).

---

2. B1–B7 as pipelines

2.0 B0 — Stakeholder-needs (the continuous-ROUND opening)

Trigger: Modeler / stakeholder, before any code exists. Drives rounds. B0 is the opening of the one continuous ROUND, not a separate phase — it seeds the graph with promised entities so that B1/B2 and every later round discharge against a non-empty frontier.

#	sub-behavior	purpose	← / ⇒	component
1	BEH-B0.voice-needs	Capture each stakeholder need as an OPEN promise (requirement / goal / success_criteria) owned by the voicing Actor.	← human dialogue; ⇒ Promise (OPEN)	PromiseEngine

#	sub-behavior	purpose	← / ⇒	component
2	BEH–B0.seed–graph	Emit the seed graph of promised entities — System + Actors + top Capabilities, all OPEN, none grounded.	⇒ seed Generation	Seeder, GenerationStore
3	BEH–B0.→round	Hand the seed graph to B–ROUND; the human oracle stays active early and fades as artifacts ground the graph.	drives B–ROUND	ExpansionKernel

B0 locks. (1) Output = a seed graph of promised entities (OPEN, none grounded). (2) Thin, coarse ROUND-seeding — the high-level goal is robust and order-insensitive; fine detail is deferred to the confluent ROUND. (3) Artifacts-first, dialogue-to-fill-gaps. (4) Termination = the VoI test with human-attention as the budget. (5) A need = an OPEN promise owned by the voicing Actor. (6) No hard B0→B1 boundary — one continuous ROUND, the human oracle active early then fading, no freeze.

2.1 B1 — Extract (B-ROUND to promise-fixpoint)

Trigger: Modeler, or Target System first-ingest. Drives rounds. Extract is not a linear pre-pass — it is B-ROUND run to promise-fixpoint: top-down intent promises meet bottom-up grounding, and scan survives only as the code-scan fact source feeding ground. A1 decomposition is emergent from the promise-DAG, not a privileged phase; downward termination is a structural fixpoint per branch (atomic leaves stop; non-atomic branches recurse). The sub-behaviors below are the fact-source + seeding feeders into the round, not an ordered batch.

#	sub-behavior	purpose	← / ⇒	component
1	BEH–B1.scan	Code-scan fact source: pure-AST walk of the target tree; emit the observable-fact inventory (one of four oracles, no privileged axis).	← Target source; ⇒ IF–ScanReport/Manifest	SourceScanner
2	BEH–B1.derive–promises	Turn facts into seed nodes carrying kind-valence open promises.	⇒ Promise nodes	PromiseEngine, Seeder
3	BEH–B1.cluster	Group modules by affinity into candidate components (emergent A1, not a fixed phase).	—	PatternDetector

#	sub-behavior	purpose	← / ⇒	component
4	BEH−B1.link	Derive edges (realizes / depends-on / exposes …) between clusters.	—	PatternDetector
5	BEH−B1.containerize	Wrap clusters as components allocated to layers.	—	Seeder
6	BEH−B1.verify	Chain-closure check of the seed graph.	⇒ verify report	GateEvaluator
7	BEH−B1.publish	Emit the gen-0 Generation.	⇒ GEN−∗	GenerationStore
8	BEH−B1.→round	Feed all residual open promises into B−ROUND to promise-fixpoint.	drives B−ROUND	ExpansionKernel

Conservation laws (emergent A1). A1 contains is a strict partition — a module is the partition atom; cross-cutting concerns escape via a shared component or LAY−CrossCutting. A2 realizes is many-to-many free. Re-binding is fluid until grounded, then sticky; reconcile + σ is the single binding operator for modules, attributes, and cross-generation identity (θ ≥ 0.85 bind / 0.60–0.85 ambiguous / < 0.60 new).

2.2 B2 — Author (CAP-DF ceremony **note → save → author → apply**)

Trigger: Modeler. Drives rounds. This is the human-in-the-loop promise-fill ceremony, reused verbatim from AMS CAP-DF and terminated by a round.

#	sub-behavior	purpose	← / ⇒	component
1	BEH−B2.note	Attach a free-text note to a specific promise slot (entity, attribute).	⇒ NOTE−∗ (CommentStub)	WalkEngine (promise-note meta-walk)
2	BEH−B2.save	Promote the note to a WorkOrder (fill_promise:<entity>:<attr>) + tracking Job (auto-advanced to queued).	⇒ WO−∗, JOB−∗	WalkEngine (save meta-walk)
3	BEH−B2.author	Assemble the prompt, call the provider, wrap the response as a ModelPatch Proposal.	← IF−LLMProvider; ⇒ PROP−∗	LLMProviderRouter, authoring expander
4	BEH−B2.apply	Validate + apply the proposal to the graph (apply_model_patch).	mutates ModelGraph	ExpansionKernel (Tier-A admit/bind)

#	sub-behavior	purpose	← / ⇒	component
5	BEH-B2.→round	Re-enter B-ROUND so the filled promise's downstream promises discharge.	drives B-ROUND	ExpansionKernel

2.3 B3 — Re-extract (incremental)

Trigger: Target System change. Drives rounds. The incremental path: only changed subtrees re-scan, identity is preserved across generations, drift is journaled.

#	sub-behavior	purpose	← / ⇒	component
1	BEH-B3.re-scan	Re-scan only changed paths (reuse lifecycle owned_paths staleness).	← changed Target paths	SourceScanner
2	BEH-B3.diff	Compute the semantic delta vs. the current Generation.	⇒ JRN-drift snapshot	GenerationStore
3	BEH-B3.identity-bind	Re-bind stable IDs via θ-crosswalk; emit gap:ambiguous-identity in the hysteresis band.	mutates IdentityStore	IdentityStore
4	BEH-B3.→round	Feed re-opened promises into B-ROUND.	drives B-ROUND; ⇒ GEN-∗	ExpansionKernel

2.4 B4 — Represent (lifecycle substrate **materialize → project → render**)

Trigger: Modeler or Downstream. Reads the converged graph; emits owned Artifacts. Forked from the lifecycle substrate.

#	sub-behavior	purpose	← / ⇒	component
1	BEH-B4.slice	Resolve a SliceSpec into a closed fragment (select + relationship closure + curate).	← IF-SliceSpec	Slicer
2	BEH-B4.materialize	Bind the fragment to a model revision + supplementary evidence refs.	—	Slicer

#	sub-behavior	purpose	← / ⇒	component
3	BEH–B4.project	Run the named projector → DiagramSpec; stamp freshness + provenance.	← IF–ViewSpec	ProjectorRegistry
4	BEH–B4.render	Deterministically render to bytes (md / html / svg / zip / ai-context).	⇒ ART–∗ (+ provenance sidecar)	Renderers, Viewer
5	BEH–B4.rebuild	Topologically rebuild a fan-out of artifacts (per-subsystem).	⇒ ART–∗ set	Renderers

2.5 B5 — Generate (MDD, oracle-in-loop)

Trigger: Modeler or arch-agent. Reads the graph; emits code Artifacts and parity evidence. Precondition: an oracle is obtainable (design §7 solution).

#	sub-behavior	purpose	← / ⇒	component
1	BEH–B5.scaffold	Emit module/class/function skeletons from component + interface specs.	⇒ ART–∗(code)	CodeGenerator
2	BEH–B5.fill–bodies	Fill bodies from body_hint / constants / test_contracts.	—	CodeGenerator
3	BEH–B5.oracle–loop	Run the oracle, feed failures back, converge.	← oracle	CodeGenerator
4	BEH–B5.verify–parity	Diff behavior vs. oracle; score parity.	⇒ JRN–test; verifies edges (A7)	ParityChecker

> Integration = grounding the dynamic interface promise (no new pipeline). Every interface (and behavior) carries two promises: a static one (left-of-Vee, grounded by reconcile + schema/model-check) and a dynamic one (right-of-Vee, grounded by the runtime oracle, the 4th fact source — surfaced in B5's verify–parity and B7's gate). An ungrounded dynamic promise is gap:integration–mismatch. Failure localizes to the interface's own promise — the connected components keep their groundings (non-monotonicity dissolved). "Component A was wrong" = an under-specified promise superseded → a diachronic Generation change, not a synchronic retraction. Integration is therefore no longer deferred to a right-of-Vee phase; it is ordinary promise-discharge inside the one ROUND.

2.6 B6 — Learn (closes the adaptation loop)

Trigger: Auditor or self (post-round). Drives rounds — the only behavior that feeds rule changes back into the engine, closing the loop AMS left open.

#	sub-behavior	purpose	← / ⇒	component
1	BEH-B6.observe	Append outcomes to the four journals (fail-soft).	⇒ JRN-gates/drift/test/llm	FeedbackJournals
2	BEH-B6.lesson	Distill deduplicated Lessons from the journals (fixes AMS "lessons overwritten").	⇒ LSN-∗	LessonStore
3	BEH-B6.tune	Adjust expander / scheduler rules from lessons.	affects ExpanderRegistry, BudgetGovernor (A4)	RuleTuner
4	BEH-B6.→round	Re-enter B-ROUND under the new rules.	drives B-ROUND	ExpansionKernel

2.7 B7 — Gate (a round stage run standalone)

Trigger: Auditor. Is B-ROUND.gate invoked on the current graph without growing it — a pure verification pass.

#	sub-behavior	purpose	← / ⇒	component
1	BEH-B7.run-checks	Evaluate the 3 verification classes (existence → gap:missing; prohibition → gap:violation; parametric → fitness).	← CON-∗	GateEvaluator
2	BEH-B7.emit	Emit GateEvent + typed gaps; never mutate the graph.	⇒ JRN-gates, GateEvent	GateEvaluator

Behavior total ≈ 36 = 6 BEH-ROUND.∗ + 1 BEH-ROUND head + 8 top-level (BEH-B0…B7) + ~22 fan-out sub-behaviors enumerated above (3 B0 + 8 B1 + 5 B2 + 4 B3 + 5 B4 + 4 B5 + 4 B6 + 2 B7, de-duplicated against the round stages they delegate to). B0 is the continuous-ROUND opening (§2.0); it shares the one engine, not a separate phase.

---

3. Comparison matrix vs. the 5 existing AMS pipelines

Per existing pipeline: what openAG does with it (fork / reuse / replace), the mapped openAG target, and the rationale.

#	AMS pipeline	stages	openAG disposition	maps to	rationale
1	Legacy 10-stage `pipeline/`	observe → infer → allocate → relate → specify → contract → validate → decompose → synthesize → emit	REPLACE (drop)	—	Linear batch, single-shot, no fixpoint, no gap-honesty. Superseded by B-ROUND. The only pipeline openAG drops.
2	v3 7-stage `pipeline_v3/`	scan → derive_promises → cluster → link → containerize → verify → publish	FORK	`B1 Extract` (§2.1)	Already promise-native; becomes the seed stage that feeds `B–ROUND`. Verify/publish reused as-is.
3	Walk 6-stage `walks/`	select → traverse → classify → rollup → evidence → emit	REUSE (as a kernel capability)	`CAP–model.traverse` / WalkEngine; and structurally ≅ the operator (§4)	The walk pipeline is the operator's shape. openAG keeps it verbatim as the declarative traversal engine and recognizes it as the crosswalk anchor.
4	Lifecycle substrate	ModelSlice → materialize → project → render → rebuild_artifacts	FORK	`B4 Represent` (§2.4) + Projection layer	The single artifact substrate. Projectors become registered pure projectors; renderers reused for byte-determinism + provenance sidecars.

#	AMS pipeline	stages	openAG disposition	maps to	rationale
5	CAP-DF ceremony	note → save → author → apply	REUSE	`B2 Author` (§2.2)	The human-in-the-loop promise-fill path. Reused verbatim; `apply` = `apply_model_patch` (Tier-A admit/bind). Terminated by a round.

Net: openAG drops exactly one pipeline (the legacy linear batch) and absorbs the other four as behaviors/capabilities feeding or reading the single `B-ROUND` fixpoint. Nothing else is lost (zero-loss crosswalk, entity-model §11).

---

4. Operator crosswalk

The central claim: openAG's one-step operator, the AMS walk pipeline, and the **`model.*`** L3 functions are the same six-stage shape viewed three ways. This is why the walk engine can be reused as the operator's declarative skeleton and why `B-ROUND` is just the batched operator.

operator (design §3.2)	walk stage (`walks/`)	`model.*` L3 functions (entity-model §3.2)	round stage (§1)	half (seam §3.4)
select	`select`	`select-promise`; legality-filter, budget-read, prioritize-VoI (neural), bundle+clamp	govern	SYMBOLIC frame + neural VoI
propose	`traverse` (hop)	`propose-candidates`	grow	NEURAL
ground	`evidence`	`gather-evidence`, `check-grounding`, `score-confidence`	ground	NEURAL
admit	`classify`	`admit-schema`, `admit-constraints`	gate	SYMBOLIC
reconcile	`rollup`	`dedup`, `identity-bind(θ)`, `roll-up-aggregate`, `canonicalize`	reconcile	SYMBOLIC

operator (design §3.2)	walk stage (`walks/`)	`model.*` L3 functions (entity-model §3.2)	round stage (§1)	half (seam §3.4)
bind	`emit` (cards)	`cache-decision`, `record-generation`	bind	SYMBOLIC

Reading the rows.

- select ↔ walk.select ↔ govern. The walk's node-selection is promise-selection; openAG wraps it with the BudgetGovernor's critical- path schedule so the batched round picks a worklist, not one promise.
- propose ↔ walk.traverse. A walk "hop" along a typed edge is exactly an expander proposing a candidate edge along an `axis`. This is the neural seam's first half — the only place a model may invent graph structure, and it never writes directly.
- ground ↔ walk.evidence. Both attach typed evidence; openAG's taxonomy (static / observational / executable) refines the walk's evidence stage and is the neural seam's second half.
- admit ↔ walk.classify. The walk's classify DSL (`has:`/`missing:`/`count:`/`verified:`…) is the gate's existence/prohibition/parametric predicate language. Verdicts become admit/reject + typed gaps.
- reconcile ↔ walk.rollup. The walk's rollup aggregation is the closure-operator join: dedup + θ identity-bind + roll-up + canonicalize.
- bind ↔ walk.emit. The walk emits Cards; openAG emits the committed `Generation` (and memoized `Decision`). Same terminal "materialize the result" step.

  Consequence. Because the shapes coincide, the existing 13 declarative WalkSpecs + 6-stage runner are not throwaway — they are the executable prototype of openAG's operator. Implementing the operator = generalizing the walk runner so `classify` consults the constraint set and `rollup` runs the θ-crosswalk. (Implementation deferred per design §12.)

---

5. Companion documents

- `2026-10-04-openAG-graph-native-engine-design.md` — formal core, the 7 axes, the one-step operator + round + fixpoint, the neurosymbolic seam, the 6 invariants, attribute algebra, critique + 4 solutions, N1/N2, and the AMS fork manifest (§9).
- `2026-10-04-openAG-entity-model.md` — the gen-0 entity ledger: full enumeration, non-uniform L3+ functional hierarchy, the I/O-entity family with ownership chains (§8), and the schema-`$defs` zero-loss reconciliation appendix (§11).